

-  Secrets
-  ABAP
-  Ansible
-  Apex
-  AzureResourceManager
-  C
-  C#
-  C++
-  CloudFormation
-  COBOL
-  CSS
-  **Dart**
-  Docker
-  Flex
-  Go
-  HTML
-  Java
-  JavaScript
-  JCL
-  Kotlin
-  Kubernetes
-  Objective C
-  PHP
-  PL/I
-  PL/SQL
-  Python
-  RPG
-  Ruby
-  Scala
-  Swift
-  Terraform
-  Text
-  TypeScript
-  T-SQL
-  VB.NET
-  VB6
-  XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules **115**  Bug **15**  Code Smell **100**

Tags ▾

Impact ▾

Clean code attribute ▾

Search by name... 🔍

Widget constructors should have a key parameter		Code Smell
"SizedBox" should be used to add a whitespace to a layout		Code Smell
Flutter widget "createState" should only return a new "State"		Code Smell
@immutable classes should only have const constructors		Code Smell
Const constructors should be invoked with const		Code Smell
Web-only libraries should only be used in Flutter web plugins		Code Smell
Generic function type syntax should be preferred for parameters		Code Smell
Null-aware assignments should make sense		Code Smell
Unnecessary use of "toList" with spread operator		Code Smell
Unnecessary braces in string interpolation should be removed		Code Smell
Initializing formals should be used		Code Smell

Widget constructors should have a key parameter

Analyze your code

Intentionality - Complete Maintainability 

 Code Smell  Major 

Widget constructors should have a **key** parameter, forwarded to the superclass constructor.

- Why is this an issue?
- How can I fix it?
- More Info

The **Widget constructor** takes a key as a named parameter. However, such a parameter is optional:

```
class Widget {
  Widget({ Key? key }) { ...}
}
```

This means that widgets inheriting from it can be created without a key parameter.

```
class MyWidget extends Widget {
  MyWidget() : super(); // No key parameter forwarded
}
```

The same applies to the **StatefulWidget** and the **StatelessWidget**.

They both define constructors with an optional key parameter, which:

- is forwarded to the `Widget` constructor when specified
- is not forwarded when omitted

That results in a widget with a `null` key.

While this is not a problem in itself, it is a good practice to always provide a key parameter to widgets, as it can be useful for debugging, testing, and performance optimizations.

For example, when a widget needs to potentially be rebuilt, Flutter compares the new widget with the old one to determine what has changed. If the widgets have the same key, Flutter assumes that they are the same, and it will not perform the rebuild. This can be useful when building the widget is expensive, or when the it has a state that should be preserved across rebuilds.

Scenarios where keys are useful include preserving the scroll position of a list, the state of a form, or the content of an editing collection.

Available In:

sonarcloud 

